

Concorrencia em Django

Views assincronas, ASGI e event loop - modulo EBAC



Django
6.0.7



Python
3.12+



Servidor
ASGI + Uvicorn



Concorrencia
Event loop / asyncio

SUMARIO

Concorrencia em Django

1. Glossario de termos
2. Aula 1 - Introducao ao Django Async
3. Aula 2 - Chamadas bloqueantes e nao bloqueantes
4. Aula 3 - Alternativas antes e depois da versao 3.1
5. Aula 4 - ASGI e o protocolo scope / receive / send
6. Aula 5 - Views assincronas e o exercicio do contador
7. Arquitetura completa - do cliente ao banco de dados
8. Exercicio complementar - fire-and-forget com `create_task`
9. Perguntas frequentes desta sessao de estudo

Glossario de termos

Referencia rapida - leia antes das aulas ou consulte durante

Blocking (bloqueante)

Uma chamada que trava a thread inteira ate ter uma resposta. Nada mais roda naquela thread enquanto ela espera.

Non-blocking (nao bloqueante)

Uma chamada que retorna na hora, mesmo sem resposta pronta. Quem chamou precisa checar depois se ja terminou (polling).

Assincrono (async)

Uma chamada que retorna na hora e avisa via callback/await quando o resultado fica pronto, sem polling manual.

Event loop

Mecanismo de uma unica thread que gerencia varias operacoes de I/O pendentes ao mesmo tempo, alternando entre elas sem bloquear.

Corrotina (coroutine)

Uma funcao 'async def' - so executa de fato quando um event loop a agenda e roda.

await

Ponto onde uma corrotina devolve o controle pro event loop e fica suspensa ate o resultado esperado chegar.

ASGI

Asynchronous Server Gateway Interface - protocolo padrao entre servidor e aplicacao assincrona em Python. Sucessor do WSGI.

WSGI

Web Server Gateway Interface - protocolo padrao entre servidor e aplicacao sincrona em Python.

scope / receive / send

As tres pecas do contrato ASGI: metadados da conexao, funcao para receber eventos, funcao para enviar eventos.

Lifespan

Tipo de scope ASGI que avisa a aplicacao quando o processo inteiro sobe ou desce (nao uma request - o worker inteiro).

Uvicorn

Servidor ASGI - implementa o event loop e fala o protocolo ASGI do lado do servidor. Pacote independente do Django.

Gunicorn

Gerente de processos WSGI (ou ASGI, via worker class Uvicorn). Decide quantos workers sobem, reinicia se um cair.

Worker

Um processo (ou thread) que atende requisicoes. Sob Uvicorn, cada worker roda seu proprio event loop persistente.

sync_to_async

Adaptador do Django/asgiref que roda codigo sincrono numa thread separada, permitindo chama-lo com 'await'.

ORM assincrono

Metodos como aget(), acreate() - eliminam a necessidade de sync_to_async manual, mas ainda usam thread por baixo na maioria dos backends.

Celery

Sistema de fila de mensagens assincrona. Roda em processo(s) separado(s) do servidor web, conectado por um broker (Redis/RabbitMQ).

Channels

Extensao do Django para protocolos alem do HTTP request/response - WebSockets, conexoes persistentes. Exige ASGI.

Load balancer

Componente que distribui requisicoes entre varias instancias identicas da aplicacao, antes mesmo de chegar no Nginx/servidor.

Primary / replica (banco de dados)

Padrao comum em bancos: um no Primary aceita escrita (fonte da verdade), replicas aceitam so leitura.

Lock (trava de banco)

Mecanismo do banco que impede duas transacoes de mexerem na mesma linha ao mesmo tempo - escopado ao dado, nao ao sistema inteiro.

I/O-bound

Tarefa cujo gargalo e esperar por rede, disco ou banco - o candidato ideal pra ganhar com async.

CPU-bound

Tarefa cujo gargalo e processamento puro (calculculo). Async nao ajuda aqui - o caminho e multiprocessing.

Aula 1 - Introducao ao Django Async

Blocking, non-blocking, event loop, ASGI e HTTPX

Por que isso importa

Uma requisicao HTTP tipica se divide em: DNS lookup e conexao TCP (fora do seu controle), request/response HTTP, tempo de resposta do servidor e renderizacao no cliente. Boa parte desse tempo total e turf do backend - e e exatamente ai que concorrencia (async) ganha performance.

Django Sync para Async

Ate a versao 3.1 (2020), o Django era exclusivamente sincrono. A partir dai, passou a suportar views assincronas nativamente - a escolha e por view, nao para o projeto inteiro.

Blocking, non-blocking e assincrono

- Blocking / sincrono: a chamada trava a thread inteira ate ter resposta.
- Non-blocking: a chamada retorna na hora, mas exige polling manual pra saber quando terminou.
- Assincrono: a chamada retorna na hora e notifica via await/callback quando o resultado chega - sem polling.

Event loop

Uma unica thread que gerencia varias operacoes de I/O pendentes ao mesmo tempo. Ideal para: acessar arquivo, operar banco de dados, chamadas de rede. Nao ajuda em tarefas CPU-bound (calculado pesado) - ali o event loop fica preso executando o calculo e nao pode alternar pra outra tarefa.

ATENCAO

Se a tarefa e CPU-bound, async nao ajuda. O caminho e multiprocessing (varios processos, cada um com seu proprio interpretador), nao asyncio.

Celery como alternativa - antes e depois

Antes da 3.1, quem precisava de assincronia usava Celery: fila de mensagens rodando em worker(s) externos, conectados por um broker (Redis/RabbitMQ). O ponto importante e que Celery e async views resolvem problemas diferentes: Celery tira o trabalho da request inteiramente (outro processo); async views evitam bloquear a thread durante I/O dentro da propria request.

NOTA

Celery não foi 'substituído' pelas async views. Em 2026, os dois convivem: async views para I/O dentro do ciclo da request, Celery para trabalho que precisa sobreviver a um restart do processo web (email, jobs longos).

ASGI e HTTPX

- ASGI (Asynchronous Server Gateway Interface): especificação sucessora do WSGI, com suporte a channels e outros protocolos além do HTTP request/response.
- HTTPX: cliente HTTP para Python com suporte síncrono e assíncrono, e a HTTP/1.1 e HTTP/2. Substitui o 'requests' dentro de views assíncronas.

Atualização de mercado (2026)

ATUALIZAÇÃO 2026 - ORM ASSÍNCRONO

O ORM do Django, na época da apostila original (3.1/3.2), era totalmente síncrono - toda query dentro de uma view async precisava de `sync_to_async()`. Hoje (Django 6.0), existem métodos assíncronos nativos (`aget`, `acreate`, etc.), que eliminam a necessidade de `sync_to_async` manual na maioria dos casos - mas por baixo dos panos, a maior parte dos backends ainda executa a query de verdade numa thread separada, não com um driver de banco nativamente assíncrono. `sync_to_async` virou exceção, não regra - mas não desapareceu.

Aula 2 - Chamadas bloqueantes e nao bloqueantes

Da teoria ao codigo real

Blocking (bloqueante / sincrono)

blocking_exemplo.py

```
import requests

def buscar_dados():
    resposta = requests.get("https://api.exemplo.com/dados") # thread trava aqui
    return resposta.json()

# 3 chamadas seguidas = SOMA dos 3 tempos
buscar_dados(); buscar_dados(); buscar_dados()
```

Assincrono (non-blocking + orquestracao)

async_exemplo.py

```
import asyncio
import httpx

async def buscar_dados():
    async with httpx.AsyncClient() as client:
        resposta = await client.get("https://api.exemplo.com/dados")
        return resposta.json()

async def main():
    # as 3 chamadas rodam CONCORRENTEMENTE
    resultados = await asyncio.gather(
        buscar_dados(), buscar_dados(), buscar_dados(),
    )

    asyncio.run(main())
```

3 chamadas concorrentes levam aproximadamente o tempo da mais lenta, nao a soma das tres - porque enquanto uma espera resposta de rede, o event loop ja esta processando as outras.

Aplicando numa view Django

views.py

```
# Sincrona (bloqueante)
def status_view(request):
    resposta = requests.get("https://api.externa.com/status")
    return JsonResponse(resposta.json())

# Assincrona
async def status_view(request):
    async with httpx.AsyncClient() as client:
        resposta = await client.get("https://api.externa.com/status")
    return JsonResponse(resposta.json())
```

ATENCAO - O ERRO MAIS COMUM

Erro classico: misturar biblioteca bloqueante dentro de 'async def'. Isso trava o event loop INTEIRO, nao so a request - pior que uma view sincrona normal, porque ali o Django tem um pool de threads pra absorver o bloqueio; num event loop de thread unica, bloquear ali para tudo que esta rodando naquele worker. Regra pratica: dentro de 'async def', toda chamada de I/O tem que ser 'await' de algo async nativo - nunca requests, nunca open() sincrónico, nunca query ORM sem o prefixo 'a'.

Quando usar cada modelo

Situacao	Modelo recomendado
CPU-bound (calculado pesado, imagem)	Nenhum dos dois - multiprocessing
Uma chamada externa isolada, view simples	Sincrono e aceitavel
Multiples chamadas de I/O independentes	Async com asyncio.gather
Alto volume de conexoes concorrentes (chat, live)	Async, servido via ASGI

Aula 3 - Alternativas antes e depois da 3.1

O que dava pra fazer, e o que mudou de verdade

Antes da 3.1 (pre-agosto/2020)

- Celery: tira o trabalho da request inteiramente, jogando pra worker externo, numa fila.
- Threading manual: abrir thread dentro de view sincrona. Risco: ORM e conexoes de banco nao sao thread-safe do jeito ingenuo - vazamento de conexao e facil.
- Workers gevent/eventlet no Unicorn: monkey-patch que reescreve funcoes bloqueantes pra virarem cooperativas. Ganha concorrência sem tocar no código Django, mas é magia escondida, difícil de debugar.
- Escalar na força bruta: aceitar o bloqueio e por mais workers/instancias atras de um load balancer. Simples de operar, ainda comum hoje.

Depois da 3.1

- `async def` nativo nas views.
- ASGI + Channels: WebSockets e conexoes persistentes - impossivel de fazer bem com WSGI puro.
- ORM assíncrono (`aget`, `acreate`) - com a ressalva do `sync_to_async` por baixo dos panos.
- Bibliotecas async nativas: `HTTPX` no lugar de `requests`, `aiofiles` no lugar de `open()`.

O que de fato mudou

Antes da 3.1, dava pra fazer concorrência - só que cada caminho era uma ferramenta por fora do Django, com seu próprio vocabulário. Depois, o framework ganhou um vocabulário nativo (`async/await`) integrado ao resto - middleware, ORM, testes. A vantagem não é 'agora é possível', é 'agora é padronizado, em vez de gambiarra por fora'.

Regua de decisao (2026)

Situacao	Ferramenta
Trabalho precisa sobreviver a restart do processo web	Celery
I/O dentro do ciclo da propria request	<code>async def view</code>
Conexao persistente, tempo real	Channels / WebSocket
Só quer aliviar throughput sem reescrever nada	Escalar workers/instancias

MISTURAR NO MESMO PROJETO?

Nada disso é mutuamente exclusivo dentro de um mesmo projeto. A decisão é por VIEW/tarefa, não pra aplicação inteira - exceto Channels, que exige que TODO o projeto rode sob ASGI (não há meio-termo por view). Celery fica de fora dessa restrição: roda em processo próprio, convive com WSGI ou ASGI sem diferença.

Aula 4 - ASGI para gerenciamento de requisicoes

Abrindo a caixa preta: scope, receive, send

O contrato real: um unico callable assincrono

contrato ASGI minimo

```
async def application(scope, receive, send):  
    ...
```

Todo servidor ASGI (Uvicorn, Daphne, Hypercorn) so precisa de uma funcao com essa assinatura pra funcionar. E o equivalente ASGI do que `application(environ, start_response)` e pro WSGI - so que assincrono e com duas 'portas' de comunicacao.

- `scope`: dicionario com metadados da conexao (tipo `http/websocket/lifespan`, metodo, `path`, `headers`). Montado uma vez, no inicio.
- `receive`: funcao `await-avel` pra PEGAR o proximo evento (corpo chegando em pedacos, aviso de desconexao).
- `send`: funcao `await-avel` pra MANDAR um evento de volta (inicio da resposta, corpo, fim).

Exemplo minimo, sem Django

`app_asgi_minimo.py`

```
async def application(scope, receive, send):  
    assert scope["type"] == "http"  
    await send({  
        "type": "http.response.start",  
        "status": 200,  
        "headers": [[b"content-type", b"text/plain"]],  
    })  
    await send({"type": "http.response.body", "body": b"oi"})
```

Onde o Django entra

`core/asgi.py`

```
import os  
from django.core.asgi import get_asgi_application  
  
os.environ.setdefault("DJANGO_SETTINGS_MODULE", "core.settings")  
application = get_asgi_application()
```

`get_asgi_application()` devolve exatamente um callable (`scope`, `receive`, `send`). Por dentro, o Django olha `scope['type']`, roda o roteamento normal (`urls.py`, `middleware`, `view`) e traduz a resposta de volta pros eventos `http.response.*`. O Django e, ele mesmo,

uma aplicacao ASGI gigante escrita sobre esse contrato minimo.

Lifespan: startup e shutdown do processo

`lifespan_exemplo.py`

```
async def application(scope, receive, send):
    if scope["type"] == "lifespan":
        while True:
            message = await receive()
            if message["type"] == "lifespan.startup":
                # abrir pool de conexao, carregar cache
                await send({"type": "lifespan.startup.complete"})
            elif message["type"] == "lifespan.shutdown":
                await send({"type": "lifespan.shutdown.complete"})
            return
```

NOTA

`scope['type'] == 'lifespan'` avisa quando o PROCESSO INTEIRO sobe ou desce - nao uma request. E o gancho certo pra abrir pool de conexao de banco uma vez por worker, nao uma vez por request. A especificacao ganhou `scope['state']` (2026), um espaco pra guardar essas coisas e o servidor garante repassar (copia rasa) pra cada request/response subsequente.

O que isso muda na pratica

Voce nao escreve `scope/receive/send` na mao no dia a dia - o Django cobre isso. O valor de entender essa camada e reconhecer o vocabulario quando aparecer: 'ASGI lifespan protocol appears unsupported' nos logs do Uvicorn, ou alguem mencionando 'scope' numa conversa tecnica.

Aula 5 - Views assincronas com Django Async Views

Function-based, class-based e o exercicio do contador

Function-based view

views.py

```
from django.http import JsonResponse

async def status_view(request):
    post = await Post.objects.aget(id=1) # ORM async nativo
    return JsonResponse({"titulo": post.titulo})
```

Class-based view

views.py

```
from django.views import View
from django.http import JsonResponse

class StatusView(View):
    async def get(self, request):
        post = await Post.objects.aget(id=1)
        return JsonResponse({"titulo": post.titulo})
```

O Django detecta automaticamente se qualquer metodo handler e 'async def' - nao precisa marcar nada especial em as_view().

Testando com AsyncClient

test_views.py

```
import pytest
from django.test import AsyncClient

@pytest.mark.asyncio
async def test_status_view():
    client = AsyncClient()
    response = await client.get("/status/")
    assert response.status_code == 200
```

O exercicio - contador de tempo

contador/views.py

```
import asyncio
import time
from django.http import JsonResponse

async def contador_view(request):
    """Aguarda N segundos sem bloquear o event loop."""
    segundos = int(request.GET.get("segundos", 3))
    inicio = time.monotonic()

    await asyncio.sleep(segundos)

    decorrido = time.monotonic() - inicio
    return JsonResponse({
        "aguardado_segundos": segundos,
        "decorrido_real_segundos": round(decorrido, 2),
    })
```

contador/urls.py

```
from django.urls import path
from . import views

urlpatterns = [
    path("", views.contador_view, name="contador"),
]
```

O teste que prova o conceito

Sobe com runserver (WSGI) e abre duas abas em /contador/?segundos=5 quase ao mesmo tempo - a segunda só começa a contar depois que a primeira termina, porque cada async view sob WSGI ganha um event loop descartável por request. Depois, sobe com Uvicorn (uvicorn core.asgi:application) e repete o teste - as duas terminam juntas, ~5s, não ~10s.

ATENÇÃO

runserver já é multi-thread por padrão no Django - então o resultado sequencial observado na prática costuma vir de as duas abas não terem disparado no EXATO mesmo instante, mais do que de uma limitação estrutural do runserver em si. Para um teste mais limpo, dispare dois curl no mesmo comando: curl ... & curl ... & wait.

Fluxo de commit

terminal

```
git checkout -b feature/async-counter-view
git add blog/views.py blog/urls.py
git commit -m "feat(blog): adiciona view assíncrona de contador de tempo"
git push origin feature/async-counter-view
```

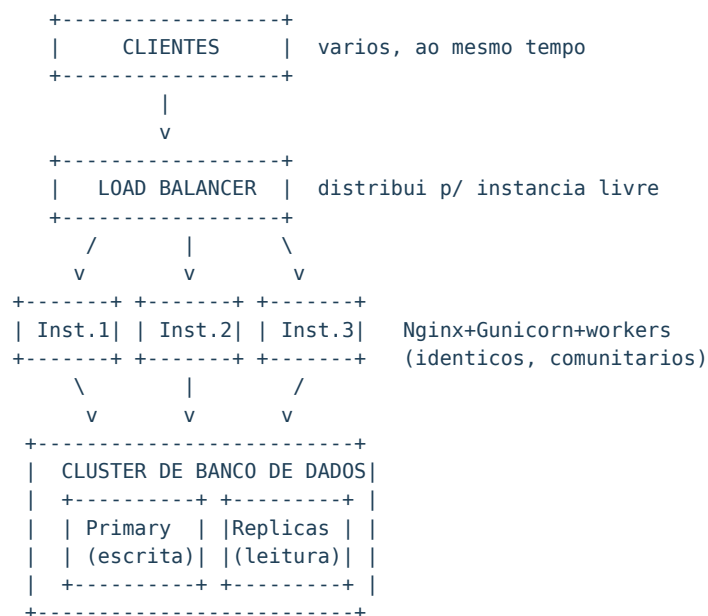
Arquitetura completa

Do clique no navegador ao commit no banco de dados

A cadeia completa

Nginx não tem event loop próprio no sentido que discutimos aqui - ele é só o 'salão', comunitário, que recebe a conexão e decide o que fazer com ela. Gunicorn é o gerente de processos: sobe workers, reinicia se um cair. Se esses workers forem do tipo UvicornWorker, cada worker é um processo Uvicorn - com um event loop persistente rodando dentro dele, com o código Django importado na mesma memória.

Escala horizontal: nada é reservado a um cliente



Alocacao por requisicao, nao por cliente

Não existe vínculo permanente entre um cliente e um worker. Cada requisição pega qualquer worker livre da pool no momento - pode ser o mesmo do request anterior, pode ser outro qualquer. Login não abre uma 'cozinha' nova - só confere uma chave (sessão/token) verificada a cada vez que a requisição cai numa raia qualquer que estiver livre.

ATENCAO

Um bloqueio mal colocado (biblioteca síncrona dentro de 'async def') afeta quem mais estiver compartilhando aquele worker específico naquele instante - não só quem causou o bloqueio. É o motivo de 'quantos workers você sobe' ser uma decisão de capacidade tão crítica em produção.

O banco de dados tem concorrência própria

Diferente da camada de aplicação (onde as N cópias são idênticas e intercambiáveis), o banco geralmente tem um único no Primary que aceita escrita - evita que duas cópias diverjam sobre o valor real de um dado. Réplicas de leitura são cópias sincronizadas com pequeno atraso, usadas só para consultas.

`transferencia_exemplo.sql`

```
BEGIN;  
SELECT saldo FROM contas WHERE id = 123 FOR UPDATE;  -- trava só ESSA linha  
UPDATE contas SET saldo = saldo - 100 WHERE id = 123;  
UPDATE contas SET saldo = saldo + 100 WHERE id = 456;  
COMMIT;  -- libera a trava
```

NOTA

O lock protege a LINHA específica sendo alterada, até o COMMIT - não o banco inteiro, nem o sistema inteiro. Duas transferências na MESMA conta esperam uma a outra; transferências em contas DIFERENTES rodam em paralelo, sem qualquer espera. A segurança vem do escopo do lock, não de 'processar uma coisa de cada vez'.

Outros estilos de arquitetura (para não generalizar demais)

- Microsserviços: em vez de uma aplicação inteira fazendo tudo, vários serviços pequenos e especializados, cada um com seu próprio load balancer/workers e às vezes banco próprio.
- Orientado a eventos (Kafka/RabbitMQ): a requisição vira mensagem numa fila, processada quando der - muda como o tempo entra na equação.
- Serverless: não existe worker persistente - cada requisição sobe uma função do zero e ela morre depois.

Cada estilo troca uma coisa por outra - consistência por disponibilidade, latência por desacoplamento, simplicidade por escalabilidade granular. 'Saudável' depende do problema que você resolve, não é uma meta única que todo estilo atinge igual.

Exercicio complementar

Fire-and-forget com asyncio.create_task

Esse exercicio (dado pelo professor do curso) demonstra um conceito DIFERENTE do exercicio do contador: resposta instantanea + trabalho continuando em background - nao 'resposta que demora mas libera o servidor pra outros'.

contador/views.py (versao corrigida)

```
import asyncio
from time import sleep
import httpx
from django.http import HttpResponse

_background_tasks = set() # segura referencia forte ate a task terminar

async def http_call_async():
    for num in range(1, 6):
        await asyncio.sleep(1)
        print(num)
    async with httpx.AsyncClient() as client:
        r = await client.get("https://httpbin.org/")
        print(r)

def http_call_sync():
    for num in range(1, 6):
        sleep(1)
        print(num)
    r = httpx.get("https://httpbin.org/")
    print(r)

async def async_view(request):
    loop = asyncio.get_running_loop()
    task = loop.create_task(http_call_async())
    _background_tasks.add(task)
    task.add_done_callback(_background_tasks.discard)
    return HttpResponse("Non-blocking HTTP request")

def sync_view(request):
    http_call_sync()
    return HttpResponse("Blocking HTTP request")
```

DUAS CORRECOES NECESSARIAS

`asyncio.get_event_loop()` esta com os dias contados - desde o Python 3.10, prefira `asyncio.get_running_loop()` dentro de uma corrotina, mais explicito. Alem disso, uma task criada com `create_task()` e guardada pelo loop so como REFERENCIA FRACA - sem uma referencia forte em algum lugar (o `set()` acima), o garbage collector pode destruir a task antes dela terminar. E um bug real, documentado, e facil de deixar passar batido.

Comparando os dois exercicios

	contador_view (nosso)	async_view (professor)
O cliente espera?	Sim, ate o resultado sair	Nao, resposta e imediata
O que fica concorrente	Requisicoes de clientes diferentes	A propria resposta e o trabalho em background
Mecanismo por baixo	Mesmo event loop, mesmo processo	Mesmo event loop, mesmo processo
Risco se der erro	Exception sobe normalmente pro cliente	Exception e engolida silenciosamente

O QUE OS DOIS TEM EM COMUM

Por baixo, e o MESMO motor: um unico event loop, multiplexando todas as corrotinas pendentes daquele processo - nao importa se e 'responder a request X', 'responder a request Y' ou 'task de background disparada pela request X'. A unica coisa que muda de um exemplo pro outro e qual corrotina o cliente esta esperando.

Esse padrao de fire-and-forget nao sobrevive a um restart do worker - e so um agendamento dentro do event loop daquele processo especifico, sem persistencia. Pra trabalho que precisa sobreviver (email transacional importante, por exemplo), o caminho correto e Celery, nao um `create_task` cru.

Perguntas frequentes desta sessao

Correcoes e esclarecimentos que surgiram estudando

O event loop e a propria aplicacao?

Nao. O event loop e infraestrutura - o mecanismo que gerencia I/O pendente. A aplicacao (Django, suas views) roda POR CIMA dele, submetendo tarefas pro loop cuidar. Sao camadas diferentes.

O Uvicorn 'vive dentro' do Django?

Nao, sao dois projetos completamente separados. Django fornece o contrato ASGI (a funcao application em asgi.py); Uvicorn e quem implementa o lado servidor desse contrato - abre a porta de rede, gerencia o event loop, chama a application do Django.

Cada worker tem um event loop rodando 'do lado de fora' dele?

Nao. Sob Uvicorn, o worker E o processo Uvicorn - o event loop roda DENTRO dele, e o codigo Django e importado na MESMA memoria daquele processo. Nao ha viagem nem mensageiro entre 'worker' e 'Uvicorn' - sao a mesma coisa.

Um cliente logado tem uma 'cozinha' (worker) so pra ele?

Nao. A alocao de worker e por REQUISICAO, nao por cliente. Nao existe vinculo permanente - cada request pega qualquer worker livre da pool naquele instante.

Numa transferencia bancaria, so trava a 'cozinha' desse cliente unico?

Nao exatamente - quem trava e o LOCK NO BANCO, escopado a linha especifica (a conta) sendo alterada, nao ao worker nem ao sistema inteiro. Um bloqueio de CODIGO mal colocado (biblioteca sincrona dentro de async def), esse sim, afeta todo mundo compartilhando aquele worker especifico - nao so quem causou.

Sync dentro de um event loop se comporta diferente de sync fora dele?

O comportamento da chamada bloqueante em si e identico - sempre trava a thread que a executa. O que muda e o RAIOS DE ESTRAGO: bloquear dentro de uma async def trava a casa inteira do event loop compartilhado, afetando todo mundo que dependia dele - nao so a propria chamada.

Async 'trava e espera' a resposta, so que sem travar os outros?

O termo certo e SUSPENDE, nao trava. No await, a corrotina para de executar naquele ponto e sai da fila - nao fica gastando processamento 'esperando ativamente'. O event loop roda outra coisa nesse meio tempo, e retoma essa corrotina exatamente onde parou quando o resultado chega.

Celery e async views sao a mesma coisa de concorrencia?

Nao. Celery roda em PROCESSO SEPARADO, conectado por fila de mensagens - nao ha event loop coordenando nada ali. Async views compartilham o MESMO processo e o mesmo event loop do servidor web. No exemplo do email de boas-vindas, o que parece

instantaneo e so o INSERT no banco (rapido por natureza) e o ato de publicar a tarefa na fila (tambem rapido) - nao os dois rodando 'juntos' no mesmo event loop.

Por que a arquitetura e sempre replicada (varios 'restaurantes')?

Porque nao existe maquina unica que escale infinitamente. E mais barato e mais resiliente ter N copias identicas da aplicacao atras de um load balancer do que uma maquina gigante - se uma copia cair, as outras seguem atendendo. O banco geralmente foge dessa regra na escrita (Primary unico) por causa de consistencia, nao por limitacao tecnica.

O que realmente quebra na escala, entre thread-por-conexao e event loop?

Nao e o custo pontual (1ms de criar um loop descartavel). E que o numero de THREADS DE SISTEMA OPERACIONAL cresce junto com o numero de conexoes no modelo de thread-por-conexao - cada thread custa memoria real. No modelo de event loop, o numero de threads fica fixo e pequeno, nao importa se sao 10 ou 10 mil conexoes - so o numero de corrotinas dentro do loop cresce, e corrotina e muito mais barata que thread.